Evaluation und Implementierung einer robusten Netzwerkarchitektur zur Echtzeit-Übertragung von Sensordaten zwischen einem ESP32-Client und einem Raspberry Pi-basierten IoT-Stack

Projektarbeit

für die

T1000

des Studiengangs Informatik

an der Dualen Hochschule Baden-Württemberg Stuttgart

von

Viktoria Backhaus

07.09.2026

Bearbeitungszeitraum	01.10.2015 - 07.09.2026
Matrikelnummer, Kurs	1695387, T41000
Ausbildungsfirma	Mercedes-Benz AG, Stuttgart
Betreuer	Paul Karcher B. Eng
Gutachter	

Sperrvermerk

Der Inhalt dieser Arbeit darf weder als Ganzes noch in Auszügen Personen außerhalb des Prüfungsprozesses und des Evaluationsverfahrens zugänglich gemacht werden, sofern keine anderslautende Genehmigung der Ausbildungsstätte vorliegt.

Stuttgart, 07.09.2026

Viktoria Backhaus

Erklärung

Ich versichere hiermit, dass ich meine Projektarbeit mit dem Thema: *Evaluation und Implementierung einer robusten Netzwerkarchitektur zur Echtzeit-Übertragung von Sensordaten zwischen einem ESP32-Client und einem Raspberry Pi-basierten IoT-Stack* selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt.

Stuttgart, 07.09.2026

Viktoria Backhaus

Inhaltsverzeichnis

Abkürzungsverzeichnis **V**

Abbildungsverzeichnis **VI**

1 Einleitung **1**
1.1 Motivation . . . 1
1.2 Zielsetzung . . . 1
1.3 Abgrenzung der Aufgabenteilung . . . 1

2 Relevante Netzwerkprotokolle und Technologien **2**
2.1 Mikrocontroller . . . 2
2.1.1 Was ist ein Mikrocontroller? . . . 2
2.1.2 Besonderheiten des ESP32 . . . 2
2.1.3 Raspberry Pi . . . 3
2.2 Der DHT11 Sensor . . . 3
2.3 OSI-Referenzmodell . . . 4
2.3.1 Physikalische Schicht (Schicht 1) . . . 4
2.3.2 Sicherungsschicht (Schicht 2) . . . 4
2.3.3 Vermittlungsschicht (Schicht 3) . . . 5
2.3.4 Transportschicht (Schicht 4) . . . 6
2.3.5 Sitzungsschicht (Schicht 5) . . . 7
2.3.6 Darstellungsschicht (Schicht 6) . . . 7
2.3.7 Anwendungsschicht (Schicht 7) . . . 7
2.3.8 Einordnung in das TCP/IP-Referenzmodell . . . 7
2.4 MQTT und HTTP . . . 8
2.5 Container-Netzwerke(Docker Bridge) . . . 8
2.6 Tunneling und Reverse Proxies (Cloudflare Tunnel) . . . 8

3 Planung und Konzeption **9**
3.1 Systemarchitektur und Datenfluss . . . 9
3.2 Netzwerktopologie . . . 9
3.3 Aufbau des Sensors . . . 9
3.4 Aufbau des Gesamten Systems . . . 10
3.5 Der Plan . . . 10

4 Implementierung der Netzwerkinfrastruktur **12**
4.1 Lokale IoT-Datenübertragung . . . 12
4.2 Interne Systemkommunikation (Docker Routing) . . . 12
4.3 Externe Datenübertragung (Cloudflare Tunnel) . . . 12
4.4 Raspberry Pi als WLAN-Access Point . . . 12

5 Kritische Betrachtung und Evaluation **13**
5.1 Evaluation der autarken WLAN-Infrastruktur 13
5.2 Analyse von TCP-Verbindungsabbrüchen 13

6 Fazit und Ausblick **14**
6.1 Zusammenfassung der Ergebnisse 14
6.2 Ausblick 14

Literatur **15**

Anhang **16**
A. Code 17

Abkürzungsverzeichnis

MCU Mikrocontroller-Einheit

EEPROM electrically erasable programmable read-only memory

IoT Internet of Things

OSI Open Systems Interconnection

TCP Transmission Control Protocol

UDP User Datagram Protocol

ACK Acknowledgement

CSMA/CA Carrier Sense Multiple Access with Collision Avoidance

MQTT Message Queuing Telemetry Transport

Abbildungsverzeichnis

3.1 Schaltplan DHT11 (eigene Abbildung) 10

6.1 Anhang: Schaltplan DHT11 (eigene Abbildung) 19
6.2 Anhang: Luftfeuchtigkeitsmessreihe 20
6.3 Anhang:Temperaturmessreihe 21
6.4 Anhang: Auschnitt aus der Taupunkttabelle ([3], S.1) 21

1 Einleitung

1.1 Motivation

Warum baut ihr das? Datenerfassung im Fahrzeug, Live-Überwachung von Sensorwerten, Aufbau eines unabhängigen Netzwerks.

1.2 Zielsetzung

Was soll am Ende stehen? Ein stabiler Datenfluss vom Sensor bis zur Visualisierung, ein autarkes lokales WLAN und die externe Erreichbarkeit des Dashboards.

1.3 Abgrenzung der Aufgabenteilung

Hier klärst du den Scope. Dein Partner macht die Datenbankstruktur, das Dashboard und die Sicherheit. Du fokussierst dich auf die Transportwege: WLAN, OSI-Modell, Docker-Routing und die Tunnel-Anbindung nach außen.

2 Relevante Netzwerkprotokolle und Technologien

Zu beginn werden alle technisch und theoretisch relevanten Konzepte, Protokolle und Technologien vorgestellt und ertläutert. Es werden die benötigten Hardware-Komponenten, wie der ESP32 Mikrocontroller, der DHT11 Sensor und der Raspberry Pi beschrieben. Daraufhin wird das OSI-Referenzmodell vorgestellt, um die verschiedenen Schichten der Netzwerkkommunikation zu erklären. Anschließend werden die Protokolle MQTT und HTTP miteinander verglichen, bevor die Besonderheiten von Container-Netzwerken (Docker Bridge) und Tunneling (Cloudflare Tunnel) erläutert werden.

2.1 Mikrocontroller

2.1.1 Was ist ein Mikrocontroller?

Ein Mikrocontroller (auch Mikrocontroller-Einheit (MCU)) ist ein kompakter Computer, der aus einem einzigen Chip besteht. Er wurde speziell dafür entwickelt, Aufgaben in eingebetteten Systemen zu übernehmen, ohne dafür ein komplexes Betriebssystem zu benötigen.

Ein solcher Chip vereint in der Regel einen Prozessor, Arbeitsspeicher, einen programmierbaren Festspeicher (wie electrically erasable programmable read-only memory (EEPROM) oder Flash) sowie diverse Ein- und Ausgabeschnittstellen. Im Gegensatz zu klassischen Mikroprozessoren benötigt ein Mikrocontroller für seine Grundfunktionen keine weiteren externen Bausteine. Er ist stattdessen darauf optimiert, direkt mit der physischen Welt zu interagieren, beispielsweise durch die Steuerung von Sensoren oder Motoren (vgl. [5]).

2.1.2 Besonderheiten des ESP32

Der ESP32 ist ein leistungsfähiger Mikrocontroller der Firma Espressif und der direkte Nachfolger des ESP8266. Er verfügt über einen 32-Bit-Dual-Core-Prozessor, was ihm eine besonders hohe Verarbeitungsgeschwindigkeit verleiht.

Zu den Kernfunktionen des Chips gehört die integrierte drahtlose Kommunikation. Der ESP32 kann sich als Client mit bestehenden WLAN-Netzwerken verbinden oder selbst als Access Point ein eigenes Netzwerk aufspannen. Zudem unterstützt das Modul Bluetooth, was es für Smart-Home- und Internet of Things (IoT)-Anwendungen ideal macht.

Darüber hinaus zeichnet sich der Mikrocontroller durch einen sehr geringen Stromverbrauch aus. Im sogenannten Deep-Sleep-Modus kann die Stromaufnahme je nach verwendetem Entwicklungsboard auf unter 0,1 mA sinken, was den Chip für energieeffiziente IoT-Projekte prädestiniert. Auch für raue Umgebungen ist der ESP32 mit einem spezifizierten Betriebstemperaturbereich von -40 °C bis +125 °C robust ausgelegt.

Um diese vielfältigen Funktionen in der Praxis einfach nutzen zu können, wird der Chip meist auf Entwicklungsboards verbaut. Diese stellen neben dem reinen Modul auch die notwendige Spannungsversorgung sowie leicht zugängliche Anschlüsse bereit (vgl. [1]).

2.1.3 Raspberry Pi

Was wann etc. auch aufs OS eingehen, besonders auf den neuen Network Manager

2.2 Der DHT11 Sensor

Der DHT11 Sensor ist in der Lage, Temperatur und Luftfeuchtigkeit zu messen.

Zum Messen der Temperatur hat der DHT11 ein Negative Temperature Coefficient Thermistor. Dabei handelt es sich um einen Widerstand, welcher geringer wird je höher die Temperatur ist, da die Leitfähigkeit des Materials zunimmt.

Für die Luftfeuchtigkeit ist ein Kondensator in den DHT11 eingebaut. Dieser besteht aus zwei Elektroden, zwischen diesen befindet sich ein flüssigkeitsempfindliches Substrat. Je höher die Luftfeuchtigkeit ist, desto mehr Kapazität besitzt dieses Substrat.

Aus diesen Daten ermittelt der Sensor die tatsächlichen Werte durch eine Kalibrierung, welche durch den Hersteller vorgenommen wurde.

Er ist in der Lage Temperaturen von 0°C bis 50°C bei einer Messgenauigkeit von ± 2,0°C zu erfassen die Luftfeuchtigkeit kann im Bereich von 20% bis 90% gemessen werden, wobei die Messgenauigkeit ± 5% beträgt (vgl. [2]).

2.3 OSI-Referenzmodell

Um die Kommunikation in komplexen Netzwerken wie dem in diesem Projekt aufgebauten IoT-System zu standardisieren, wird in der Literatur häufig das Open Systems Interconnection (OSI)-Referenzmodell (Open Systems Interconnection) herangezogen. Es wurde in den 1980er Jahren von der Internationalen Organisation für Normung (ISO) entwickelt, um die Interoperabilität zwischen verschiedenen Netzwerksystemen zu gewährleisten. Es besteht aus sieben aufeinander aufbauenden Schichten, welche von Schicht 7 bis 1 beim Senden und von 1 bis 7 beim Empfang durchlaufen werden. Diese sieben Schichten erfüllen jeweils spezifische Aufgaben, die einen reibungslosen und standardisierten Datenaustausch ermöglichen. Dabei ist zu betonen, dass das Modell selbst keine spezifischen Protokolle oder Komponenten definiert, sondern lediglich die Aufgaben der jeweiligen Schichten vorgibt. Aus diesem Grund lassen sich in der Praxis genutzte Protokolle nicht immer eindeutig einer einzigen Schicht zuordnen, da sie oftmals mehrere Aufgaben gleichzeitig übernehmen (vgl. [4], S.41-42).

2.3.1 Physikalische Schicht (Schicht 1)

In der untersten Schicht, der Physikalischen Schicht, werden die rohen Bits über das Medium übertragen. Es geht also um die elektrotechnische Umsetzung der Datenübertragung heutzutage ist das häfig WLAN, bzw. die physischen Funkwellen, welche diese Schicht abdecken. Aber auch LAN oder USB-Verbindungen würden hier eingeordnet werden (vgl. [4], S.43).

2.3.2 Sicherungsschicht (Schicht 2)

Während die physikalische Schicht lediglich den Bitstrom überträgt, dient die Sicherungsschicht (Data Link Layer) der Abstraktion hardwarenaher Übertragungsfehler. Ziel ist es, der übergeordneten Vermittlungsschicht eine zuverlässige, logische Verbindung zur Verfügung zu stellen. Dieser Prozess basiert primär auf der Framing-Technik, bei der Datenströme in diskrete Einheiten segmentiert werden.

Zur Gewährleistung der Datenintegrität implementiert die Schicht Mechanismen zur Fehlererkennung sowie, je nach Protokollspezifikation, Bestätigungsverfahren Acknowledgement (ACK). Eine integrierte Flusssteuerung schützt zudem langsame Empfängerinstanzen vor einer Datenüberlastung durch den Sender. Im Kontext des ESP32 und der verwendeten WLAN-Kommunikation übernimmt die Sicherungsschicht zudem die Medienzugriffssteuerung (MAC). Diese koordiniert den Zugriff auf das geteilte Funkmedium, um Kollisionen zu

minimieren und eine strukturierte Datenübermittlung im Netzwerkverbund sicherzustellen (vgl. [4], S.43).

WLAN (IEEE 802.11) und CSMA/CA

Für die kabellose Datenübertragung im lokalen Netzwerk nutzt der ESP32 den WLAN-Standard IEEE 802.11. Da drahtlose Netzwerkschnittstellen im Gegensatz zum klassischen kabelgebundenen Ethernet (IEEE 802.3) (vgl. [4], S.20) in der Regel *half-duplex* arbeiten, also nicht gleichzeitig auf derselben Frequenz senden und empfangen können, ist eine direkte Kollisionserkennung während des Sendens physikalisch nicht möglich. Ein gesendetes Signal ist um ein Vielfaches stärker als ein empfangenes, weshalb der Sender „taub“ für Fehler ist, solange er selbst funkt.

Um Signalüberlagerungen in der Luft dennoch zu minimieren, kommt auf der MAC-Teilschicht das Protokoll Carrier Sense Multiple Access with Collision Avoidance (CSMA/CA) (Carrier Sense Multiple Access with Collision Avoidance) zum Einsatz. Bevor ein Knoten, wie in diesem Fall der Mikrocontroller, einen Datenrahmen sendet, prüft er zunächst, ob das Funkmedium frei ist (Carrier Sense). Um Kollisionen mit anderen wartenden Geräten präventiv zu vermeiden (Collision Avoidance), wartet der Sender eine zufällig gewählte Zeitspanne (Backoff-Zähler) ab, bevor er die Übertragung startet (vgl. [4], S....).

Da der Sender eigene Kollisionen nicht registrieren kann, erfordert CSMA/CA zwingend eine kurze Empfangsbestätigung ACK der Gegenstelle. Bleibt dieses ACK aus, geht der Sender von einem Übertragungsfehler oder einer Kollision aus, verdoppelt seine zufällige Wartezeit (Exponential Backoff) und unternimmt einen erneuten Sendeversuch(vgl. [4], S.303).

2.3.3 Vermittlungsschicht (Schicht 3)

Die Vermittlungsschicht (Network Layer) steuert den Betrieb des Kommunikationsnetzes und ist primär für die Wegewahl (Routing) der Datenpakete vom Sender zum Zielort verantwortlich. Während die darunterliegende Sicherungsschicht lediglich den Datenaustausch zwischen direkt physikalisch benachbarten Knoten koordiniert, ermöglicht die Vermittlungsschicht eine logische Kommunikation über mehrere Netzwerkgrenzen hinweg.

Eine der zentralen Aufgaben dieser Schicht ist die Bestimmung der optimalen Route durch das Netzwerk. Diese Routen können entweder statisch konfiguriert oder dynamisch an die aktuelle Netzwerkauslastung angepasst werden, um Überlastungen (Congestion) zu vermeiden. Sollten Datenpakete für ein zugrundeliegendes Netzwerk zu groß sein,

übernimmt diese Schicht zudem die Fragmentierung in kleinere Einheiten (vgl. [4], S.43-44).

Das dominierende Protokoll zur Umsetzung dieser Aufgaben in modernen Netzwerken ist das Internet Protocol (IP). Es arbeitet verbindungslos (connectionless), was bedeutet, dass jedes Datenpaket (Datagramm) unabhängig und ohne vorherigen Verbindungsaufbau durch das Netzwerk geroutet wird (vgl. [4], S.35-37). Es weist jedem Netzwerkknoten eine eindeutige logische IP-Adresse zu, wodurch eine zuverlässige Adressierung und Weiterleitung innerhalb komplexer Netzwerktopologien, wie beispielsweise auch in virtuellen Container-Netzwerken, gewährleistet wird (vgl. [4], S.46-47) (vgl. [4], S.438-439).

2.3.4 Transportschicht (Schicht 4)

Die grundlegende Aufgabe der Transportschicht besteht darin, Daten von den darüberliegenden Schichten entgegenzunehmen, diese bei Bedarf in kleinere Einheiten zu unterteilen (Segmentierung) und an die Vermittlungsschicht weiterzuleiten. Gleichzeitig isoliert sie die oberen Anwendungsschichten vollständig von den unvermeidlichen technologischen Änderungen der zugrundeliegenden Hardware.

Ein entscheidendes Merkmal der Transportschicht ist ihre Eigenschaft als erste echte Ende-zu-Ende-Schicht (End-to-End Layer) im OSI-Modell. Im Gegensatz zu den Schichten 1 bis 3, die als Kette logisch nur mit ihren direkten physischen Nachbarknoten kommunizieren, stellt Schicht 4 eine logische Verbindung direkt zwischen dem Ursprungs- und dem Zielsystem her. Ein Programm auf dem Quellsystem führt somit eine direkte Konversation mit dem entsprechenden Programm auf dem Zielsystem.

Zudem bestimmt diese Schicht, welche Art von Dienst (Type of Service) der darüber liegenden Schicht bereitgestellt wird. Der am häufigsten genutzte Dienst ist ein logisch fehlerfreier Punkt-zu-Punkt-Kanal, der garantiert, dass die Nachrichten in exakt der Reihenfolge ausgeliefert werden, in der sie gesendet wurden (vgl. [4], S.44).

TCP und UDP

Um die beschriebenen Dienste der Transportschicht zu realisieren, kommen in der modernen Netzwerktechnik primär zwei Protokolle zum Einsatz, die sich gegenseitig ergänzen: das User Datagram Protocol (UDP) und das Transmission Control Protocol (TCP).

UDP (User Datagram Protocol):
UDP ist ein sehr einfaches, verbindungsloses Transportprotokoll. Es nimmt Daten von der Anwendungsebene entgegen und leitet sie weiter, ohne eine logische Verbindung aufzubauen

oder eine Zustellgarantie zu übernehmen. Es wird primär für Anwendungen genutzt, bei denen eine schnelle Übertragung wichtiger ist als absolute Zuverlässigkeit, beispielsweise bei Multimedia-Streams (vgl. [**tanenbaum**], S.541).

TCP (Transmission Control Protocol):
Im Gegensatz dazu ist TCP ein verbindungsorientiertes Protokoll und ist das am weitesten verbreitete Transportprotokoll im Internet. Es garantiert die fehlerfreie und in der korrekten Reihenfolge eintreffende Auslieferung der Daten. Um diese Zuverlässigkeit zu gewährleisten, implementiert TCP, im gegensatz zu UDP, komplexe Mechanismen wie Flusskontrolle, Überlastungssteuerung (Congestion Control) und das erneute Senden verloren gegangener Pakete (Retransmissions). Da in der vorliegenden IoT-Architektur die verlässliche, lückenlose Übertragung der Sensordaten via Message Queuing Telemetry Transport (MQTT) essenziell ist, basiert die gesamte Kommunikation auf TCP (vgl. [4], S.552-559).

TCP-Verbindungsaufbau (Three-Way-Handshake):
Da TCP verbindungsorientiert arbeitet, muss vor der eigentlichen Datenübertragung zunächst ein logischer Kanal zwischen Sender (Client) und Empfänger (Server) etabliert werden. Dies geschieht über den sogenannten Three-Way-Handshake.

Dabei wartet der Server zunächst passiv auf eingehende Verbindungsanfragen auf einem spezifischen Port (Listen-Status). Möchte ein Client eine Verbindung aufbauen, sendet er ein TCP-Segment mit gesetztem SYN-Flag (Synchronize) an den Server. Existiert auf dem Zielport ein wartender Prozess, akzeptiert der Server die Anfrage und antwortet mit einem kombinierten SYN-ACK-Segment (Synchronize-Acknowledge). Abschließend bestätigt der Client den Erhalt dieser Nachricht mit einem finalen ACK-Segment (Acknowledge), womit die zuverlässige bidirektionale Verbindung erfolgreich hergestellt ist und die eigentliche Datenübertragung der Anwendungsschicht beginnen kann (siehe Abbildung **??**)(vgl. [4], S.560).

2.3.5 Sitzungsschicht (Schicht 5)

2.3.6 Darstellungsschicht (Schicht 6)

2.3.7 Anwendungsschicht (Schicht 7)

2.3.8 Einordnung in das TCP/IP-Referenzmodell

Bild aus Tanenbaum Seite 46. Und erklärung.

2.4 MQTT und HTTP

Der Vergleich!

MQTT: Leichtgewichtig, Publish/Subscribe, perfekt für den ESP32 und instabiles WLAN. HTTP: Request/Response-Modell, schwergewichtiger. Wird intern von Telegraf (um Daten an InfluxDB zu senden) und von Grafana (für das Web-Dashboard) genutzt.

2.5 Container-Netzwerke(Docker Bridge)

Wie kommunizieren Docker-Container? Erklärung des virtuellen Switches (docker0), der ein komplett eigenes Subnetz aufspannt, getrennt vom physischen WLAN.

2.6 Tunneling und Reverse Proxies (Cloudflare Tunnel)

Die Theorie, wie man ein lokales Netzwerk sicher ins Internet bringt. Erklärung, dass der Tunnel von innen nach außen aufgebaut wird, wodurch man NAT-Firewalls am Router umgeht, ohne Ports öffnen zu müssen.

3 Planung und Konzeption

3.1 Systemarchitektur und Datenfluss

Der Weg der Daten durch die OSI-Schichten. Sensor -> TCP/MQTT (WLAN) -> Raspberry Pi -> TCP/HTTP (Docker Bridge) -> Datenbank. Später dann: Grafana -> Cloudflare Tunnel -> Endnutzer.

3.2 Netzwerktopologie

Geplant war eine autarke Sterntopologie mit dem Raspberry Pi als zentralem Access Point für alle Geräte.

3.3 Aufbau des Sensors

Für den praktischen Aufbau des Systems werden folgende Komponenten benötigt:

- DHT11 - Temperatur- und Luftfeuchtigkeitssensor
- ESP32 - Mikrocontroller mit WLAN-Funktion
- Breadboard - zur schnellen, lötfreien Prototypenentwicklung
- Micro-USB-Kabel - zur Stromversorgung und Programmierung
- 3 Jumper-Kabel - zur elektrischen Verbindung zwischen Sensor und Mikrocontroller

Der Aufbau erfolgt wie in Abbildung 3.1 dargestellt. Der DHT11 wird mit drei Leitungen an den ESP32 angeschlossen: ein Datenpin, ein 3.3-V-Anschluss sowie eine Masseverbindung. Das Micro-USB-Kabel verbindet den ESP32 mit einem Computer und dient sowohl als Stromquelle als auch zur Übertragung des Programmcodes. Durch die geringe Leistungsaufnahme des Systems reicht die USB-Stromversorgung vollständig aus.

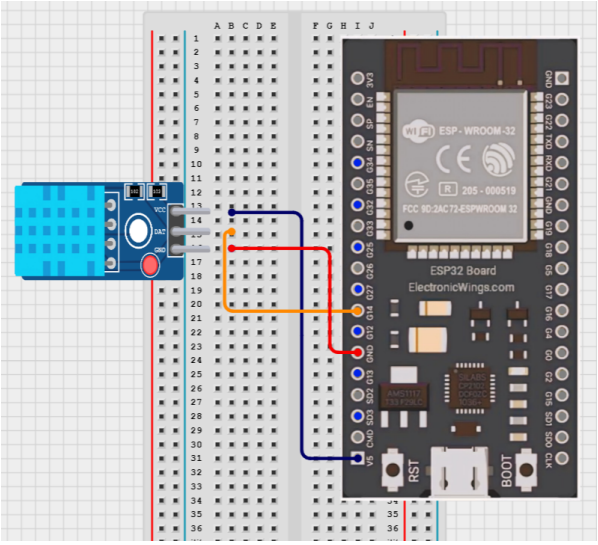

Abbildung 3.1: Schaltplan DHT11 (eigene Abbildung)

Zur Programmierung wird Visual Studio Code mit der Erweiterung PlatformIO genutzt, die Mikrocontrollerprojekte, Bibliothekenverwaltung, Kompilieren und Hochladen auf den ESP32 vereinfacht. Für eine einfache Weboberfläche zur Anzeige der Messdaten kommt zusätzlich Live Server zum Einsatz, um die Website lokal zu hosten und in Echtzeit zu aktualisieren.

Der Plan sieht vor, den DHT11 in regelmäßigen Abständen auszulesen, die Temperatur- und Feuchtigkeitswerte digital zu verarbeiten und über den seriellen Monitor zu prüfen. Optional können die Daten über die Weboberfläche dargestellt werden, um die Genauigkeit des Sensors für Anwendungen wie Defogging-Systeme zu beurteilen.

3.4 Aufbau des Gesamten Systems

Jeder hat seinen Sensor aufgebaut (Abstand, Wasser, Wasser, DHT11), diese sind über WLAN mit dem Raspberry Pi verbunden, der als zentraler Server fungiert. Der Pi wird nur mit einem Stromkabel an den Strom angeschlossen.

3.5 Der Plan

Zuerst war das ziel erstmal nur die Daten an den Pi zu senden, dafür sollten die Sensoren per MQTT die DAten an den Pi senden. Dann haben wir ertsmal die nötigen Docker auf dem Pi installiert, und die Skripte für die Sensoren angepasst, als diese gesendet haben, haben

wir die Daten in der InfluxDB gespeichert, und mit Grafana visualisiert. Dann wollten wir die Daten auch von außerhalb des lokalen Netzwerks zugänglich machen, dafür haben wir den Cloudflare Tunnel eingerichtet, damit wir https nutzen konnten, da unser nächstes Ziel die Sicherheit des Systems war, dafür haben wir auch die nötigen Berechtigungen und API-Keys in der .yaml Datei für Docker übergeben, damit die Container untereinander kommunizieren können, ohne dass Daten das virtuelle docker0-Netz verlassen müssen. Zum Schluss wollten wir den Pi auch als WLAN-Access Point nutzen, damit die Sensoren sich direkt mit dem Pi verbinden können, ohne dass ein externes WLAN benötigt wird.

4 Implementierung der Netzwerkinfrastruktur

4.1 Lokale IoT-Datenübertragung

Wie der ESP32 sich ins WLAN einwählt, die TCP-Verbindung aufbaut und die Sensordaten per MQTT an den Port 1883 des Raspberry Pis sendet.

4.2 Interne Systemkommunikation (Docker Routing)

Die Umsetzung der Theorie aus 2.5. Grafana und Telegraf finden die InfluxDB einfach über den Container-Namen (z.B. http://influxdb:8086). Daten verlassen hierfür nie das virtuelle docker0-Netz. Docker netz mit einer .yaml DAtei strukturiert, dabei benötigte berechigungen und APU-Keys Übergeben (mehr dazu im Sicherheitsteil).

4.3 Externe Datenübertragung (Cloudflare Tunnel)

Die praktische Umsetzung von 2.6. Installation des Cloudflared-Dienstes auf dem Pi, Verknüpfung mit der gekauften Domain und das automatische HTTPS-Zertifikat für den sicheren Browser-Zugriff.

4.4 Raspberry Pi als WLAN-Access Point

Das ganze Problem mir nicht genug Strom vom Netzteil, dann mit neuem Netzteil nochmal probieren. Warum? IPhone Hotspot auf 5 Verbindungen begrenzt und es ist näher an einer reellen IoT Anwendung, in einem Auto.

5 Kritische Betrachtung und Evaluation

5.1 Evaluation der autarken WLAN-Infrastruktur

Die Hotspot-Story aus der Praxis! Pi wurde per NetworkManager zum Access Point gemacht. Problem: Sende-Modus zieht zu viel Strom -> Undervolting -> Hardware stürzt ab. Fallback auf den iPhone-Hotspot. Dann nochmal probiert mit anderem Netzteil. Bei realanwendung auf nötige Stromversorgung achten, um kontinuirichen Betrieb zu gewährleisten.

5.2 Analyse von TCP-Verbindungsabbrüchen

Hier kannst du super dein Wissen aus dem Theorieteil anwenden! Diagnose des errno: 104 "Connection reset by peer"Fehlers. Wie ihr analysiert habt, dass die TCP-Verbindung auf unterster Ebene vom Host (Pi) abgelehnt wurde, weil der Broker-Container sich in einer Neustart-Schleife befand, durch komische Passwort Datei, die sich da reingemogelt hatte.

6 Fazit und Ausblick

6.1 Zusammenfassung der Ergebnisse

Das Netzwerk steht. Daten fließen von Sensoren über MQTT und HTTP sicher bis ins Internet, ohne Router-Ports öffnen zu müssen.

6.2 Ausblick

Anschaffung eines stärkeren Netzteils (min. 5.1V / 3.0A), um den Raspberry Pi in Zukunft stabil als eigenen Access Point betreiben zu können.

Literatur

[1] *ESP32 Grundlagen*. 2023. URL: https://digitalewelt.at/esp32-grundlagen. (zuletzt geprüft: 14.12.2025).

[2] *Hochschule Hamm Lippstadt: Temperatur und Feutchtigkeitssensor DHT11*. 2023. URL: https://wiki.hshl.de/wiki/index.php/Temperatur-_und_Feuchtigkeitssensor_DHT11. (zuletzt geprüft: 14.12.2025).

[3] Korff Superwand. *Taupunktabelle*. pdf online. (zuletzt geprüft: 14.12.2025). n.d.

[4] Andrew S. Tanenbaum und David J. Wetherall. *Computer Networks*. 5. Aufl. Boston: Prentice Hall, 2011. ISBN: 978-0132126953.

[5] *Was ist ein Mikrocontroller*. 2025. URL: https://www.ibm.com/de-de/think/topics/microcontroller. (zuletzt geprüft: 14.12.2025).

Anhang

A. Code

B. Abbildungen

A. Code Der verwendete Code zum Auslesen der DHT11 Sensoren mit dem ESP32 und zum anzeigen der Messwerte auf einer Website.

```
#include <Arduino.h>
#include "DHT.h"
#include <WiFi.h>

#define DHTPIN 14
#define DHTTYPE DHT11

const char* ssid = "iPhone VBackhaus";
const char* password = "wtIhe357";

DHT dht(DHTPIN, DHTTYPE);
WiFiServer server(80);

void setup() {
Serial.begin(115200);

WiFi.begin(ssid, password);
Serial.println("Verbindet mit WLAN...");
while (WiFi.status() != WL_CONNECTED) {
  delay(1000);
  Serial.println("Noch keine Verbindung...");
}

Serial.println("Verbunden");
Serial.print("IP-Adresse: ");
Serial.println(WiFi.localIP());

dht.begin();
server.begin();
Serial.println("Server gestartet");
}

void loop() {
WiFiClient client = server.available();
if (!client) return;

Serial.println("Neuer Client verbunden");
String request = client.readStringUntil('\r');
Serial.println(request);
client.flush();

//pruefen ob die Anfrage nach "/data" geht
if (request.indexOf("GET /data") >= 0) {
  float h = dht.readHumidity();
```

```
      float t = dht.readTemperature();

      client.println("HTTP/1.1 200 OK");
      client.println("Content-Type: application/json");
      client.println("Connection: close");
      client.println();

      if (isnan(h) || isnan(t)) {
      client.println("{\"error\": \"Sensorfehler\"}");
      } else {
      client.printf("{\"temperature\": %.2f, \"humidity\": %.2f}", t, h);
      }
      return;
    }

    //HTML datei mit Script
    String html = R"rawliteral(
    <!DOCTYPE html>
    <html>
    <head>
      <meta charset="UTF-8">
      <title>DHT11 Sensor</title>
    </head>
    <body>
      <h1>ESP32 DHT11 Sensorwerte</h1>
      <p>Temperatur: <span id="temp">--</span> C</p>
      <p>Luftfeuchtigkeit: <span id="hum">--</span> %</p>

      <script>
      // Diese Funktion holt neue Daten vom ESP32
      async function ladeDaten() {
        const antwort = await fetch('/data');      // Anfrage an /data schicken
        const daten = await antwort.json();        // Antwort als JSON lesen
        document.getElementById('temp').textContent = daten.temperature;
        document.getElementById('temp').textContent = daten.temperature;
      }

      ladeDaten();                 // Beim Laden der Seite einmal aufrufen
      setInterval(ladeDaten, 3000); // Alle 3 Sekunden automatisch neu laden
      </script>
    </body>
    </html>
```

```
    )rawliteral";

    client.println("HTTP/1.1 200 OK");
    client.println("Content-type:text/html");
    client.println("Connection: close");
    client.println();
    client.println(html);
    client.println();
    }
```

Listing 6.1: Code zur Auslesung des DHT11 Sensors mit dem ESP32

B. Abbildungen

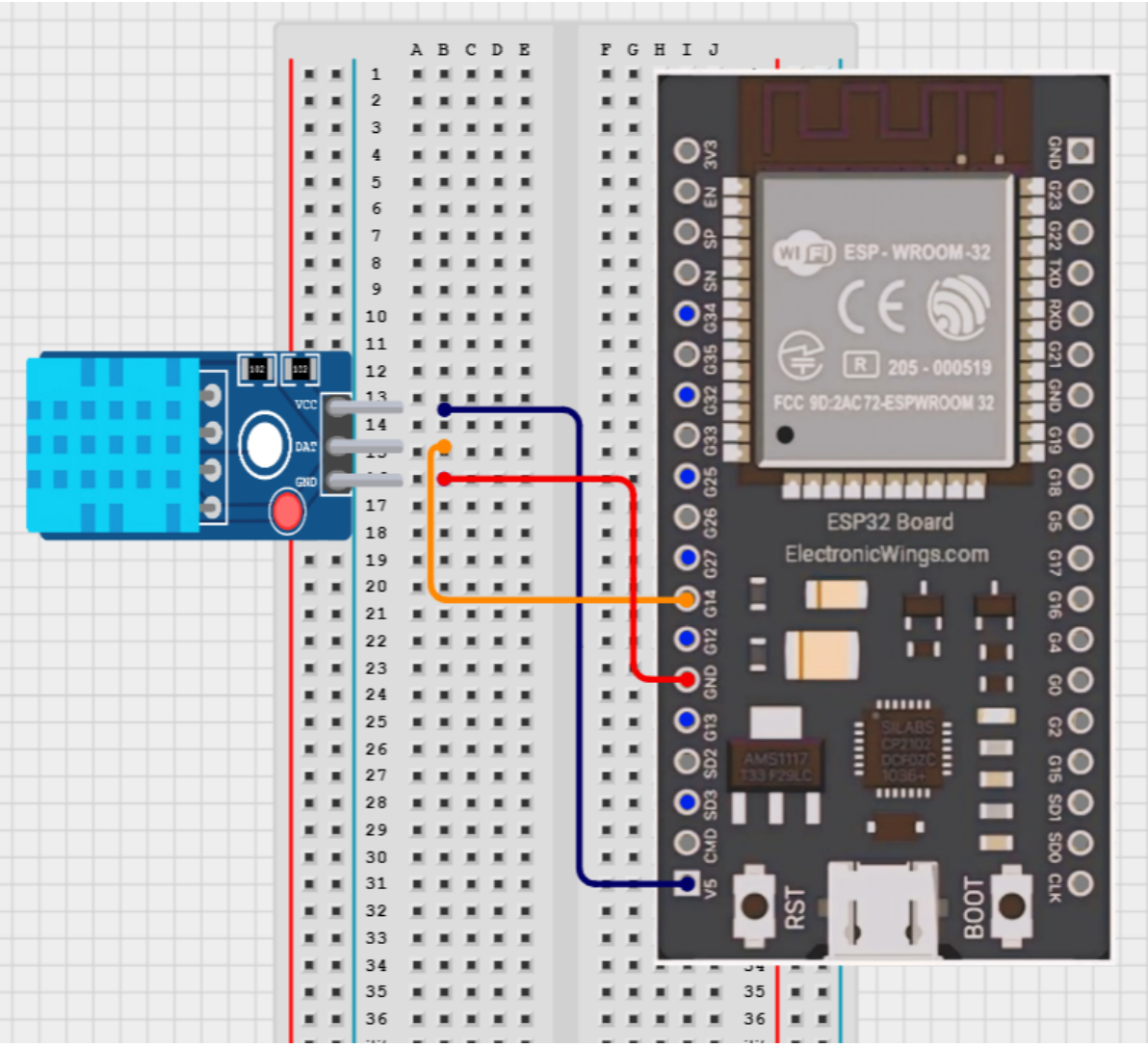

Abbildung 6.1: Anhang: Schaltplan DHT11 (eigene Abbildung)

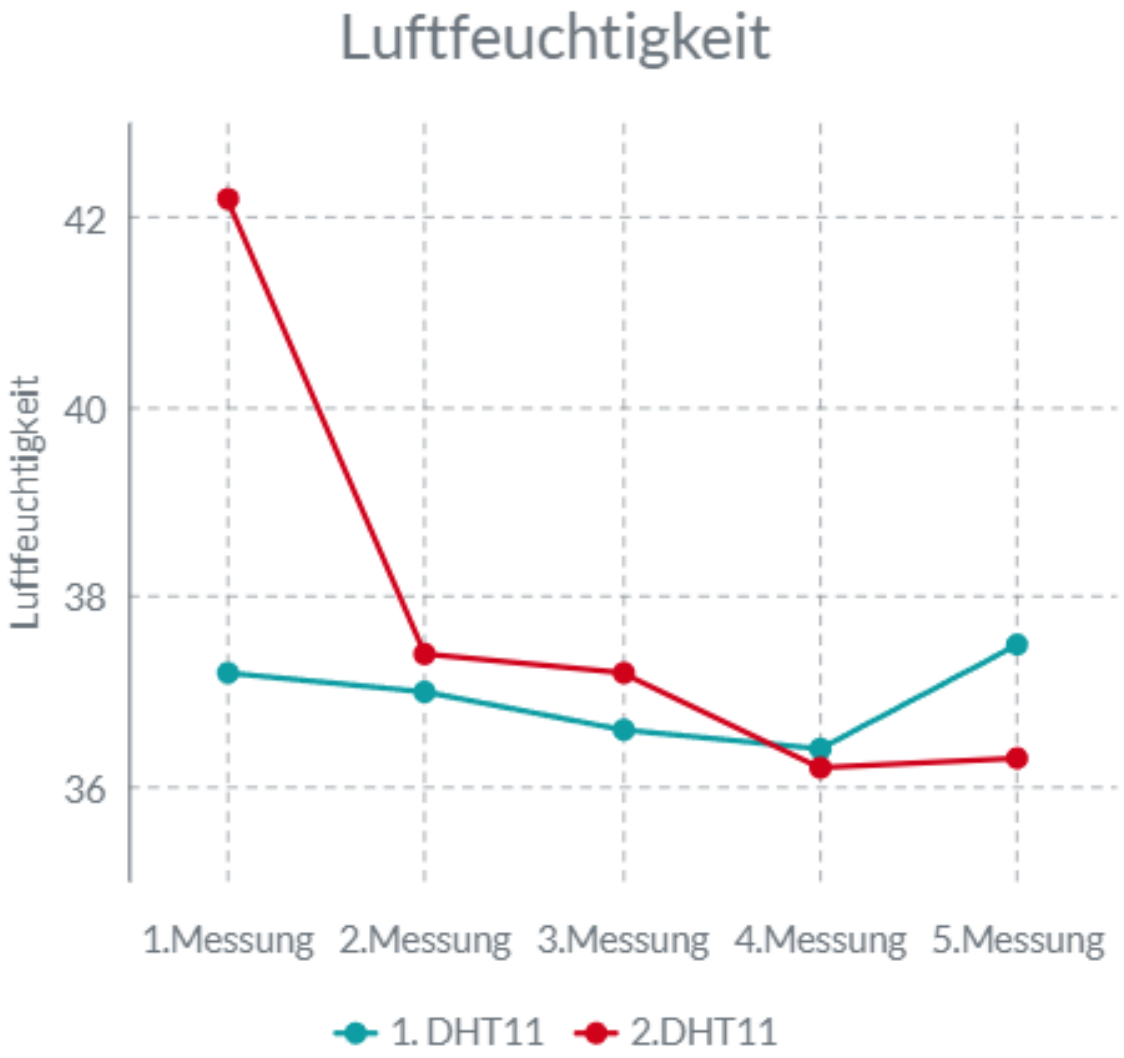

Abbildung 6.2: Anhang: Luftfeuchtigkeitsmessreihe

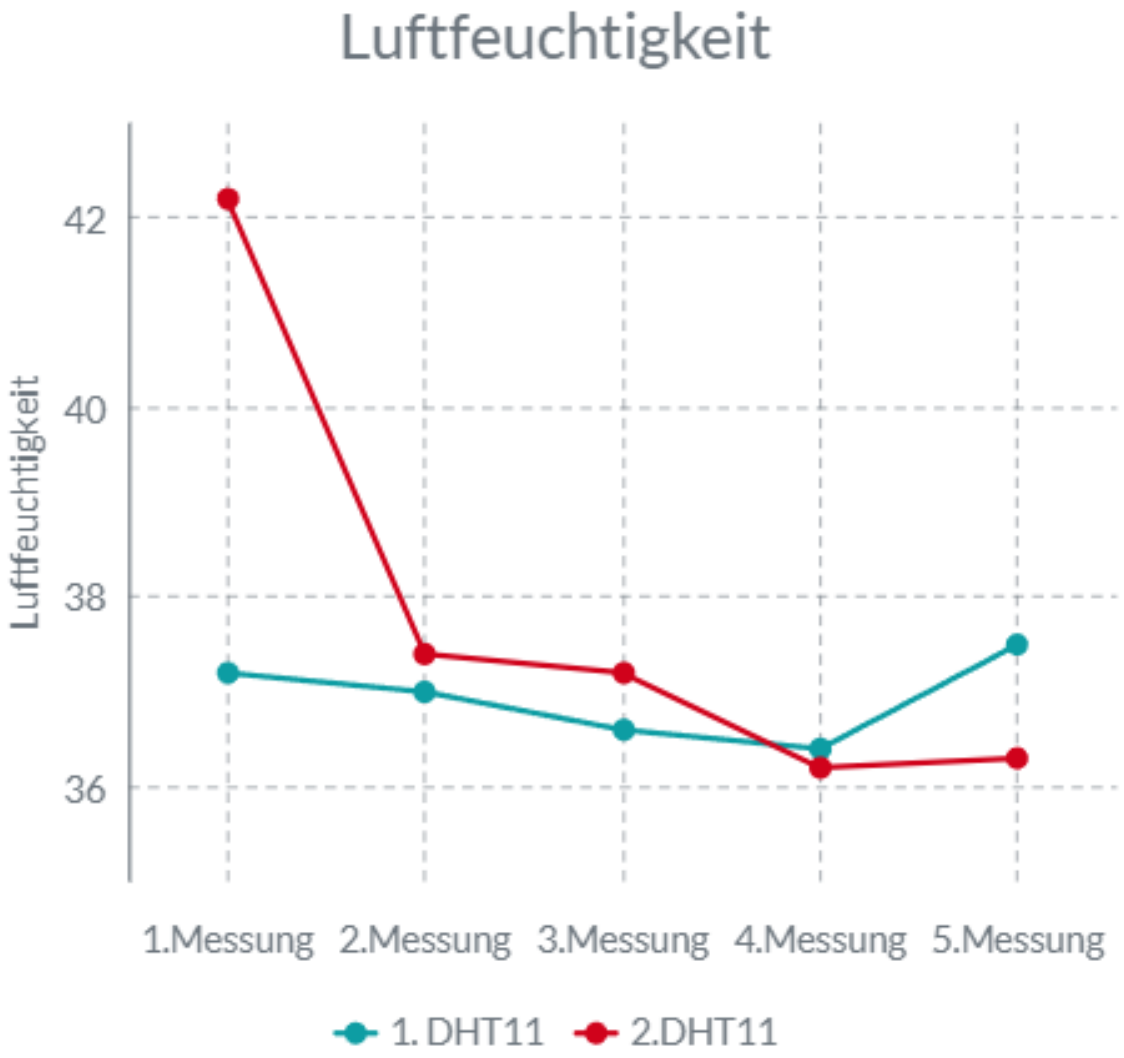

Abbildung 6.3: Anhang:Temperaturmessreihe

Luft-Temp.	Taupunkttemperaturen in °C bei einer relativen Luftfeuchtigkeit von										
C°	45%	50%	55%	60%	65%	70%	75%	80%	85%	90%	95%
2°C	-7.77	-6.56	-5.43	-4.40	-3.16	-2.48	-1.77	-0.98	-0.26	0.47	1.20
4°C	-6.11	-4.88	-3.69	-2.61	-1.79	-0.88	-0.09	0.78	1.62	2.44	3.20
6°C	-4.49	-3.07	-2.10	-1.05	-0.08	0.85	1.86	2.72	3.62	4.48	5.38
8°C	-2.69	-1.61	-0.44	0.67	1.80	2.83	3.82	4.77	5.66	6.48	7.32
10°C	-1.26	0.02	1.31	2.53	3.74	4.79	5.82	6.79	7.65	8.45	9.31

Abbildung 6.4: Anhang: Auschnitt aus der Taupunkttabelle ([3], S.1)